SIGma **Fall 2026, November 5th, 2026**

2027 CG:SHOP: multi-robot cover

Lecturer: Ian Chen *Scribe: Ian Chen*

In the previous weeks, we've been focusing on the single-robot case, with either a square-cutter or circle-cutter. There is a very rich space in determining how to split the problem so that we can load-balance across multiple-robots. Namely, as in CG:SHOP 2027, we will consider the makespan objective: minimize the maximum length. In particular, our main tool will be an approximation algorithm for TreeCover: cover a graph using k trees minimizing the maximum size of any tree.

1 multirobot forest coverage

We discuss multirobot forest coverage, from Zheng et al. [4].

1.1 problem formulation

The terrain consists of square cells, each binary (visitable or not). Each cell is weighted by its cost to visit, and is split evenly into 4 small cells. Let n be the number of visitable small cells. A cell has weight 4 in the unweighted case, and weight in $\mathcal{Z}$ in the weighted case, where the cost of a move is the average of its start and end.

Robots are non-blocking (they can occupy the same cell) and are sized to small cells. We are given an initial cell for each robot. The objective is make-span, with or without return (path vs cycle).

The problem is NP-hard, by reduction from 3-partition: split an array of length $3m$ into m sets, all of equal weight and of size 3.

2 existing work

2.1 spanning tree coverage (STC)

STC [2] is for a single robot: take any spanning tree of the large cells, and circumnavigate it. This is optimal in the unweighted case, and in the weighted case is off by an additive term of the largest weight.

2.2 multi-robot spanning tree coverage (MSTC)

MSTC [3] improves the path that circumnavigates the spanning tree. Each robot walks the arc from the circumnavigation in the same direction (given start point), say counterclockwise, as in Fig. 1.

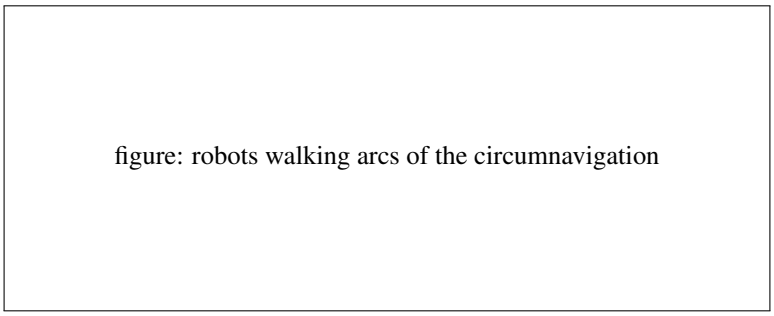


Figure 1: **MSTC arcs:** TODO.

1. We can choose start points, without return. Then, it is very easy to subdivide evenly (with small additive error).
2. We can choose start points, with return. Then, it is not easy (say, a 2-approx naively).
3. Fixed start positions (say, unweighted, with k robots). If a stretch is longer than $n/2$, then we have the others assist ($k \geq 3$), as in Fig. 2.

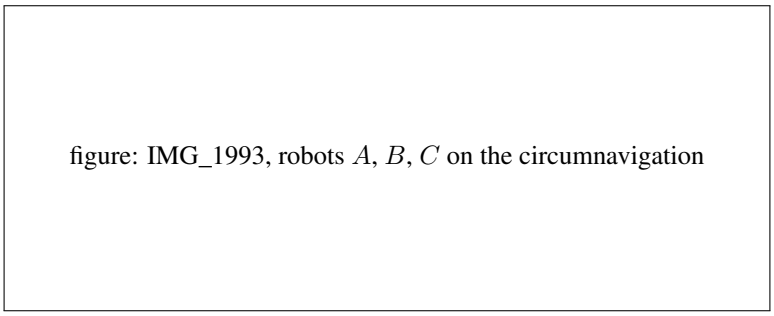


Figure 2: **Assisting a long stretch:** TODO.

Proposition: The cost is at most $n/2$, which is the best for a tree in the worst case (all stacked).

4. Weighted (then stretch the paths accordingly).

3 this work

This work bounds the approximation ratio (previous work has no bound). Take the graph of large cells, and create k roots (suppose they are distinct).

- forest cover gives a 4-approx ($4(M+1)$, with $M+1$ nodes), by induction on the number of nodes, and trees $\leq$ paths
- a 4-approx for the forest cover algorithm gives a 16-approx (with return and without)
- with $\epsilon = k(w_{\max}/w_{\text{sum}})$, there is a $4(1+\epsilon)$ approx for tree cover; the novelty is for weighted tree cover (small modifications)

4 4-approx unweighted tree cover

So, we give a 4-approx for unweighted tree cover [1] (G connected).

- **input**: unweighted graph G, with distinct roots $r_1, \ldots, r_k$
- **output**: trees $T_1, \ldots, T_k$ (not necessarily disjoint), minimizing the maximum size

4.1 aside: max-flow min-cut theorem

We formulate the optimization problem on a digraph G with $V = \{s, t\} \sqcup V'$, where s is the source and t is the sink, and with edge capacity c. A flow assigns $f : E \to [0, c(e)]$ to each edge.
Proposition: Any flow $\leq$ any cut (weak duality).
Proof: Consider only the edges crossing $(S, \bar{S})$.

Equality is known as strong duality, and is more interesting. Consider for each edge $u \to v$ the residual edge $v \to u$, where

$$w(u \to v) = c(u \to v) - f(u \to v)$$
$$w(v \to u) = f(u \to v)$$

See Fig. 3 for an example. That is, the weight is the amount of flow we can change the flow by.

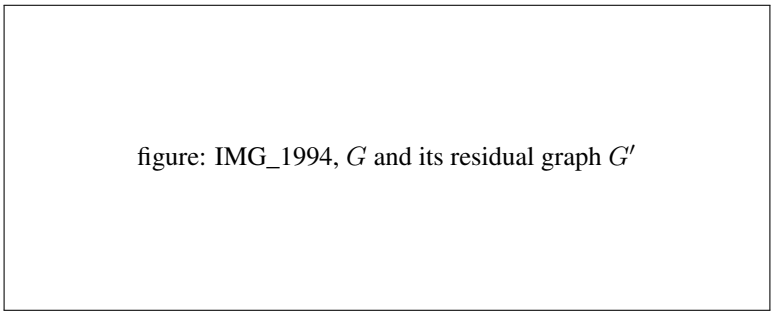


Figure 3: **Residual graph:** TODO.

Proposition: Max flow $\implies$ no s, t path in G'.

Therefore, we have an (s, t)-cut, the value is the same, and we achieve that max-flow = min-cut.

> **Exercise 4.1:** Why can we assume that there is a max-flow using only integers, when the capacities are integers?

Flows are very useful in many graph problems, e.g. matchings/scheduling.
Corollary: There exists an L-matching in a bipartite graph $(L \cup R, E)$ with $|L| \leq |R|$ if $\forall S \subseteq L$, $|N(S)| \geq |S|$.
Proof: Consider the graph H in Fig. 4, where the edges in the middle have cost $|L|$. Then, there exists a min-cut $(\{s\}, L \cup R \cup \{t\})$, because replacing $S \cap L$ and moving that and its neighbors to T does not increase the cut.

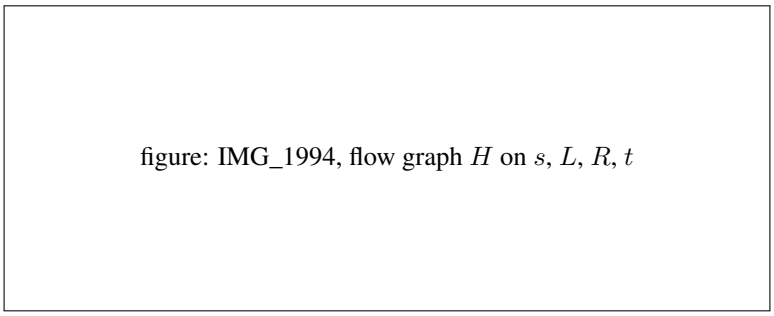


Figure 4: **Matching as a flow:** TODO.

4.2 the approx algorithm

Now, we are ready to give the approx algorithm. Namely, it is sensible to:

1. start from a forest
2. break apart "large" trees, and move them to another root

To determine "large", we parameterize based on a parameter $B \in [\![|V|]\!]$. The size of a tree is its number of vertices. A branch at v is v together with some of its child subtrees, and a chunk is a smallest branch with size $> B$. In particular, while a tree has size $> B$, we extract a chunk, keeping its top v in the tree, and reassign.

Ending this process, we have the chunks and the roots, as in Fig. 5.

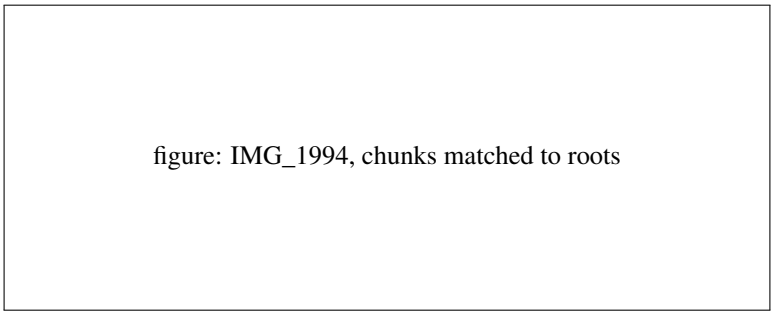


Figure 5: **Matching chunks to roots:** TODO.

We take a min-max matching of the chunks to the roots, where the weight is the distance from the chunk to the root. Then, the total cost is

$$\max_{i \in [\![k]\!]} \{|\text{leftover } i| + |\text{chunk matched to } i| + \text{dist}(i)\}$$

Observe that the number of chunks is at most $|V|/B$, so we should set $B \geq |V|/k$ to ensure such a matching exists.

Now, we need to choose B and to bound the cost. For this, we can simply try every B (a linear search) and take the one with the minimum cost. In particular, for the analysis, the algorithm is no worse than setting $B = B^*$, the optimal.

$$\max_{i\in[\![k]\!]}\{\ldots\} \leq \underbrace{\max(\text{leftover})}_{\leq B^*} + \underbrace{\max(\text{chunk})}_{\leq 2B^*+1} + \underbrace{\max(\text{dist in matching})}_{\leq B^*-1} \leq 4B^*$$

This holds as, restricting to only edges with weight $\leq B^* - 1$, each subset S (of chunks) has at least $|S|$ neighbors (pigeonhole: consider the roots covering S in the optimal).

There is a small approx-factor gap in the (vertex-)weighted case.

solutions

> **Solution to Exercise 4.1: TODO.**

References

[1] G. Even et al. "Min-max tree covers of graphs". In: *Operations Research Letters* 32 (2004), pp. 309–315.

[2] Y. Gabriely and E. Rimon. "Spanning-tree based coverage of continuous areas by a mobile robot". In: *Annals of Mathematics and Artificial Intelligence* 31 (2001), pp. 77–98.

[3] N. Hazon and G. Kaminka. "Redundancy, efficiency, and robustness in multi-robot coverage". In: *Proceedings of the International Conference on Robotics and Automation*. 2005, pp. 735–741.

[4] Xiaoming Zheng et al. "Multirobot Forest Coverage for Weighted and Unweighted Terrain". In: *IEEE Transactions on Robotics* 26.6 (2010), pp. 1018–1031. DOI: 10.1109/TRO.2010.2072271.